Prob 1. (40 pts total) Translate the following C function to ASM functions.

```c
int32_t x_cmp_y_load(int32_t x, int32_t y, int16_t *ptrA, int32_t *ptrB) {
    if (x > y) {
        return (int32_t)*(++ptrA) * 2;
    } else {
        return *(ptrB++) / 2;
    }
}
```

Note that we need to write the assembly function to strictly implement the C statements. For example, even if the change of the pointers cannot go outside of the function, we still implement their changes in assemble.

1. (20 pts) Rewrite the above C function in assembly with the following start code:

```
x_cmp_y_load_pl:    @ PL stands for positive logic
    cmp     r0, r1
    b..     x_cmp_y_load_pl_then
x_cmp_y_load_pl_else:
    .
    .
    .
    .
    .
x_cmp_y_load_pl_then:
    .
    .
    .
    .
x_cmp_y_load_pl_end:
    bx      lr
```

where `b..` should be replaced by the appropriate instruction and `.` should be replaced by an assembly instruction. (May not be one to one.) Note that we use the positive logic (the same logic as the C `if` test) to have the `else` code block before the `then` code block. Write your code in the space provided in the code.

2. (10 pts) Rewrite the above C function in assembly with the following start code:

```
x_cmp_y_load_nl:    @ NL stands for negative logic
    cmp     r0, r1
    b..     x_cmp_y_load_nl_else
x_cmp_y_load_nl_then:
    .
    .
    .
    .
    .
x_cmp_y_load_nl_else:
    .
    .
    .
    .
x_cmp_y_load_nl_end:
    bx      lr
```

Note that we use the negative logic (the opposite logic of the C `if` test) to have the `else` code block following the `then` code block. Write your code in the space provided in the code.

3. (10 pts) Rewrite the above C function in assembly using the CEX approach with the following start code:

```
x_cmp_y_load_cex:    @ CEX: conditional execution
    cmp     r0, r1
    .
    .
    .
    .
    .
    .
    .
    bx      lr
```

Note that we use the negative logic (the opposite logic of the C `if` test) to have the `else` code block following the `then` code block. Write your code in the space provided in the code.

Solution:

Part 1:

```
x_cmp_y_load_pl:    @ PL stands for positive logic
    cmp     r0, r1
    bgt     x_cmp_y_load_pl_then
x_cmp_y_load_pl_else:
    ldr     r0, [r3], #4
    asr     r0, #1
    b       x_cmp_y_load_pl_end
x_cmp_y_load_pl_then:
    ldrsh   r0, [r2, #2]!
    lsl     r0, #1
x_cmp_y_load_pl_end:
    bx      lr
```

Part 2:

```
x_cmp_y_load_nl:    @ NL stands for negative logic
    cmp     r0, r1
    ble     x_cmp_y_load_nl_else
x_cmp_y_load_nl_then:
    ldrsh   r0, [r2, #2]!
    lsl     r0, #1
    b       x_cmp_y_load_nl_end
x_cmp_y_load_nl_else:
    ldr     r0, [r3], #4
    asr     r0, #1
x_cmp_y_load_nl_end:
    bx      lr
```

Part 3:

```
x_cmp_y_load_cex:    @ CEX: conditional execution
    cmp     r0, r1
    ittee   gt
```

```
    ldrshgt r0, [r2, #2]!
    lslgt   r0, #1
    ldrle   r0, [r3], #4
    asrle   r0, #1
    bx      lr
```

-:-

Prob 2. (60 pts, 20 pts each)

Consider a problem implementing the following math function for integer input x:

$$f(x) = \begin{cases} -10, & x < -5, \\ 10, & x \geq 5, \\ 2x, & \text{otherwise.} \end{cases}$$

Part 1: Write a C function to implement the above math function using the prototype below: `int mathf_c(int x)`. Note that you need to use a simple if-else-if structure for this problem.

Part 2: Translate the above C function to assembly using the prototype below: `int mathf_cmp(int x)`. Note that you need to use the CMP approach for this problem.

Part 3: Translate the above C function to assembly using the prototype below: `int mathf_cex(int x)`. Note that you need to use the CEX approach for this problem.

Solution:

Part 1:

```
int mathf_c(int x) {
    int signx;
    if (x < -5) {
        signx = - 10;
    } else if (x >= 5) {
        signx = 10;
    } else {
        signx = x << 1;
    }
    return signx;
}
```

Part 2:

```
@ int mathf_cmp(int x);
@ x -> r0
mathf_cmp:
    cmn     r0, #5
    bge     mathf_cmp_elseif_cmp
    ldr     r0, =-10
    b       mathf_cmp_end
mathf_cmp_elseif_cmp:
    cmp     r0, #5
    blt     mathf_cmp_elseif_else
```

```
mathf_cmp_elseif_then:
    ldr     r0, =10
    b       mathf_cmp_end
mathf_cmp_elseif_else:
    lsl     r0, #1
mathf_cmp_end:
    bx      lr
```

Part 2:

```
@ int mathf_cex(int x);
@ x -> r0
mathf_cex:
    cmn     r0, #5
    itt     lt
    ldrlt   r0, =-10
    blt     mathf_cex_end
    cmp     r0, #5
    ite     ge
    ldrge   r0, =10
    lsllt   r0, #1
mathf_cex_end:
    bx      lr
```

-:-

Prob 3. (40 pts total, 20 pts each) Given the following C function for returning a 16-bit or 32-bit value using a pointer:

```
int32_t load_based_on_x_c(int x, uint16_t *ptrA, uint32_t *ptrB) {
    if (x <= 20 || x >= 25) {
        return (int32_t)*++ptrA * 4;
    } else {
        return *++ptrB / 4;
    }
}
```

1. Rewrite the above C function in assembly using standard conditional branch-based approach with the function name being `load_based_on_x_cmp`.
2. Rewrite the above C function in assembly using as much conditional execution as possible with the function name being `load_based_on_x_cex`.

Solution:

Part 1:

```
@ int32_t load_based_on_x_cmp(int x, uint16_t *ptrA, uint32_t *ptrB)
@ x -> r0, *ptrA -> r1, *ptrB -> r2
load_based_on_x_cmp:
    cmp     r0, #20
    ble     load_based_on_x_cmp_then
    cmp     r0, #25
    blt     load_else
load_based_on_x_cmp_then:
```

```
    ldrh    r0, [r1, #2]!
    lsl     r0, #2
    b       load_based_on_x_cmp_end
load_based_on_x_cmp_else:
    ldr     r0, [r2, #4]!
    lsr     r0, #2
load_based_on_x_cmp_end:
    bx      lr
```

Part 2:

```
@ int32_t load_based_on_x_cex(int x, uint16_t *ptrA, uint32_t *ptrB)
@ r0 = x, r1 = ptrA, r2 = ptrB
load_based_on_x_cex:
    cmp     r0, #20
    ittt    le
    ldrhle  r0, [r1, #2]!
    lslle   r0, #2
    ble     load_based_on_x_cex_end

    cmp     r0, #25
    ittee   ge
    ldrhge  r0, [r1, #2]!
    lslge   r0, #2
    ldrlt   r0, [r2, #4]!
    lsrlt   r0, #2
load_based_on_x_cex_end:
    bx      lr
```

Note that this function has some repetiation code, which is not desired.

> -:-